

Alpaca 周频新闻回测说明

这份文档说明当前仓库里最新的 Alpaca 周频新闻回测工作流，包括：

- 周频新闻是如何抓取的
- dataset 是如何生成的
- `start_date` / `end_date` 的最新定义
- `no_signal` / `no_article_provided` 是如何处理的
- 如何用 notebook 跑这套回测

1. 整体目标

这套流程的目标是：

1. 按周为每个股票单独抓取 Alpaca 新闻
2. 每周每个股票最多抓 `a` 篇新闻
3. 每 `b` 篇新闻合并成一个 prompt 样本
4. 如果某周某股票完全没有新闻，则不调用 LLM，直接标记为 `no_signal`
5. 生成一个兼容现有 `backtest/backtester.py` 的 dataset
6. 后续可以继续调用原有的 Agent 1 / Agent 2 回测逻辑

2. 当前涉及的核心文件

- 配置文件：[alpaca_backtest.example.json](#)
- 周频 Alpaca pipeline：[alpaca_news_pipeline.py](#)
- 回测主逻辑：[backtester.py](#)
- dataset 解析：[dataset_parser.py](#)
- Colab notebook：[alpaca_weekly_backtest.ipynb](#)

3. 配置文件怎么控制流程

当前最重要的配置项如下。

```
{
  "symbols": ["GLD", "SLV", "IAU", "AMZN", "AAPL", "COP", "XLU"],
  "start": "2025-12-25",
  "end": "2026-04-30",
  "frequency": "weekly",
  "fetch_articles_per_symbol": 50,
  "combine_articles_per_sample": 5,
  "holding_period_days": 7,
  "sort": "desc",
  "limit_per_request": 50,
  "include_content": false,
  "exclude_contentless": false,
```

```
"content_max_chars": 2000,
"requests_per_minute": 180,
"request_timeout_sec": 20.0,
"max_retries": 3,
"retry_base_delay_sec": 1.0,
"cache_dir": "../output/alpaca_news_cache",
"dataset_output_path": "../output/alpaca_news_backtest_dataset.csv",
"backtest_output_path": "../output/alpaca_news_backtest_results.csv",
"batch_size": 10,
"max_rows": null,
"force_refresh": false
}
```

其中最关键的是：

- `symbols` 要抓新闻并回测的股票列表
- `start` / `end` 整个新闻抓取区间
- `frequency` 当前只支持 "weekly"
- `fetch_articles_per_symbol` 也就是你之前说的 `a` 每个股票、每个周窗口，最多抓多少篇新闻
- `combine_articles_per_sample` 也就是你之前说的 `b` 每多少篇新闻合并成一个 prompt 样本
- `holding_period_days` 控制回测评价窗口长度，但不是直接用自然日当终点，而是先算原始结束日期，再映射到美股交易日
- `sort` 当前通常设为 "desc"，表示优先拿最新新闻
- `include_content` 是否把 Alpaca 返回的正文 content 一并放入缓存和 dataset
- `content_max_chars` content 截断长度
- `force_refresh` 为 `true` 时忽略缓存，强制重新抓新闻

4. 当前周频抓取逻辑

4.1 周窗口如何切分

当前逻辑不是按自然周一到周日强制对齐，而是：

- 从 `config.start` 开始
- 每 7 天滚动切一个窗口
- 直到 `config.end`

例如：

- `start = 2026-01-03`
- `end = 2026-01-31`

则窗口会是：

1. 2026-01-03 到 2026-01-09
2. 2026-01-10 到 2026-01-16
3. 2026-01-17 到 2026-01-23
4. 2026-01-24 到 2026-01-30
5. 2026-01-31 到 2026-01-31

这些窗口会写成 `window_key`，格式如下：

```
2026-01-03__2026-01-09
```

4.2 新闻如何抓取

对于每一个：

- `symbol`
- `window_key`

pipeline 都会单独向 Alpaca 发请求。

每次请求的核心参数是：

- `symbols` = 当前股票
- `start` = 当前周窗口开始
- `end` = 当前周窗口结束
- `sort` = `desc`
- `limit` = `min(limit_per_request, remaining)`

然后：

- 最多抓 `fetch_articles_per_symbol` 篇
- 如果该周不足这个数量，就只保留实际能抓到的篇数
- 不会跨周补新闻
- 不会把别的 `symbol` 的新闻混进来

5. 新闻如何变成 prompt 样本

每个股票、每个周窗口内抓到的新闻会按以下规则处理：

1. 按 Alpaca 返回顺序保留
2. 当前配置通常是 `sort = desc`
3. 每 `combine_articles_per_sample` 篇合并成一个样本

例如：

- `fetch_articles_per_symbol` = 50
- `combine_articles_per_sample` = 5

如果某个股票在某周抓到了 12 篇新闻，则会生成 3 个样本：

- 第 1 个样本：前 5 篇
- 第 2 个样本：中间 5 篇
- 第 3 个样本：最后 2 篇

注意：

- 最后一组不足 `b` 篇也会保留
- 不会因为不满一组就丢弃

6. 没有新闻时如何处理

如果某个股票在某个周窗口里完全没有新闻：

- 不会调用 Agent 1
- 不会调用 Agent 2
- 会直接在 dataset 中写一行特殊标记

当前标记是：

- `skip_llm = True`
- `forced_signal = "no_signal"`
- `skip_reason = "no_article_provided"`
- `pass_reason = "no_article_provided"`

后续 `backtester.py` 读到这类行时，会直接输出：

- `direction = "no_signal"`
- `signal_direction = "no_signal"`

而不会再进入 LLM 推理。

7. 当前 `start_date` / `end_date` 的最新定义

这是现在最重要的一部分。

定义：

- `news_window_start` 周频新闻窗口起始日期
- `news_window_end` 周频新闻窗口结束日期
- `start_date` 写入 dataset，并被 backtester 用来做收益评价的实际开始日期
- `end_date` 写入 dataset，并被 backtester 用来做收益评价的实际结束日期

7.1 `start_date`

当前逻辑：

```
start_date = news_window_end 之后严格下一个美股交易日
```

例子：

- `news_window_end = 2025-12-31`
- `2026-01-01` 是元旦，美股休市
- 所以：

```
start_date = 2026-01-02
```

7.2 end_date

当前逻辑：

```
raw_end_date = start_date + timedelta(days=holding_period_days)
end_date = raw_end_date 当天如果是交易日就直接用，否则顺延到下一个美股交易日
```

注意这里是：

on or after

不是：

strictly after

例子：

- `start_date = 2026-01-02`
- `holding_period_days = 7`
- `raw_end_date = 2026-01-09`
- `2026-01-09` 是正常交易日

所以：

```
end_date = 2026-01-09
```

而不是 `2026-01-12`。

7.3 为什么这样设计

这个设计的含义是：

- 周窗口内的新闻先收集完
- 周窗口结束后，到下一个可交易日才开始评价
- 持有 `holding_period_days` 个自然日
- 如果终点落在休市日，再顺延到下一个交易日

这比直接用“最后一篇新闻日期”更接近“周信号形成后可执行”的逻辑。

8. 缓存机制

当前缓存文件在：

- `news_cache.json`
- `cache_manifest.json`

缓存逻辑如下：

1. 先读取 config
2. 计算 config fingerprint
3. 启动时检查缓存是否存在
4. 如果 fingerprint 一致，则直接复用缓存
5. 如果 fingerprint 变化，则重新抓取并覆盖缓存

所以你会看到类似日志：

```
Config changed since last cache build. Refreshing Alpaca news cache.
```

这表示配置文件发生了变化，例如：

- 股票池变化
- 起止日期变化
- `a / b` 变化
- `holding_period_days` 变化
- `include_content` 变化

9. CLI 如何跑这套流程

9.1 只生成 dataset，不跑模型

```
python -m backtest.run_alpaca_backtest --config  
configs/alpaca_backtest.example.json --dataset-only
```

这个命令会：

1. 读取 config

2. 检查并使用缓存
3. 按周抓取 Alpaca 新闻
4. 生成 dataset CSV

输出通常包括：

- `dataset_path`
- `article_count`
- `dataset_rows`

9.2 继续跑完整回测

```
python -m backtest.run_alpaca_backtest --config
configs/alpaca_backtest.example.json
```

这个命令会在 dataset 生成完成后继续：

1. 调用 Agent 1 提取 fingerprint
2. 调用 Agent 2 生成 signal
3. 计算 realized return
4. 输出 backtest 结果 CSV

10. Notebook 如何跑这套回测

当前 notebook 文件：

[alpaca_weekly_backtest.ipynb](#)

这个 notebook 的设计目标是：

- 先看 dataset-only 阶段是否正常
- 再决定是否启用完整 backtest

10.1 推荐运行顺序

第一步：安装依赖

Notebook 前面的 cell 会安装：

- `datasets`
- `transformers`
- `accelerate`
- `pydantic`
- `python-dotenv`
- `yfinance`
- `tqdm`
- `vllm`

第二步：挂载 Drive 并定位仓库

在 Colab 里，notebook 会：

- mount Google Drive
- clone 或 pull 仓库
- 切到项目根目录

这里要确认：

- `PROJECT_PATH`
- `REPO_URL`

是否与你实际使用的仓库一致。

第三步：配置环境变量

会设置：

- `ALPACA_API_KEY`
- `ALPACA_API_SECRET`
- `FINGPT_MODEL_PATH`
- `SHARE_SINGLE_LLM_BETWEEN_AGENTS`
- `FINGPT_CALIBRATION_T`
- `FINGPT_LOGITS_MAX_TOKENS`

如果只是先跑 dataset-only，模型路径即使暂时不完整也问题不大。

第四步：加载 config

Notebook 会读取：

[alpaca_backtest.example.json](#)

并打印：

- 股票列表
- 日期区间
- 周频参数
- `a`
- `b`
- `holding_period_days`

第五步：先跑 dataset-only

Notebook 默认先执行：

```
run_alpaca_backtest_pipeline(..., run_existing_backtest=False)
```

这一步是最推荐先检查的，因为它可以帮助你确认：

- 周频切窗是否合理
- 每周每股抓到多少新闻
- `no_signal` 行是否生成正确
- dataset 规模是否符合预期

第六步：检查 cache 和 dataset

Notebook 后面的 cells 会检查：

- cache manifest
- cache 内容样例
- dataset 前几行
- `skip_llm` 行数
- `pass_reason` 分布
- `no_signal` 样本

这一步特别适合调试：

- 为何某些股票没有新闻
- 某周是否抓满 `a` 篇
- dataset 行数为什么比想象中多或少

第七步：开启完整回测

Notebook 里有一个开关：

```
RUN_FULL_BACKTEST = False
```

改成：

```
RUN_FULL_BACKTEST = True
```

之后再执行对应 cell，才会真的调用：

- Agent 1
- Agent 2
- backtester

第八步：查看 metrics

最后一个部分会读取：

- backtest output CSV

并调用：

```
compute_metrics(...)
```

打印回测指标。

11. 推荐使用方式

建议你每次都按这个顺序操作：

1. 修改 config
2. 先跑 `dataset-only`
3. 检查 cache 和 dataset
4. 确认 weekly window、`a`、`b`、`no_signal` 都正确
5. 再开启完整 backtest

这样能避免一上来就跑大模型，结果发现 dataset 构造逻辑不对。

12. 当前这一版逻辑的总结

一句话总结当前工作流：

先按周为每个股票单独抓 Alpaca 新闻，再按篇数合并成 prompt 样本；没有新闻的周直接标记为 `no_signal`；真正写入回测的 `start_date` 和 `end_date` 使用美股交易日规则而不是自然日。

当前最核心的几点是：

- 抓新闻是按周、按股票分别抓
- 每周每股最多 `a` 篇
- 每 `b` 篇合并成一个模型样本
- 没新闻不跑模型
- `start_date` 和 `end_date` 采用“交易日驱动”的规则

如果你后面还想扩展，这份文档建议优先补充的方向有：

- 是否改成自然周对齐
- 是否支持多种 `end_date_mode`
- 是否把美股交易日逻辑抽成单独模块
- 是否在 notebook 里增加每周/每股新闻数量可视化